QuakeCore 2026: SW4 Performance Consultancy

Progress report, 22 September – 29 October 2026

Alexander Pletzer, Geoffrey Weal (REANNZ) and
Jake Faulkner (QuakeCore/University of Canterbury)

29 October 2026

Abstract

This report summarises the work carried out for QuakeCore under the REANNZ consultancy "QuakeCore 2026" since its start on 22 September 2026. The consultancy targets the single-precision port of the seismic wave propagation code SW4 (`ucgmsim/sw4`, branch `single_precision`). Over five weeks we validated the single-precision port, established the best compiler, build and runtime configuration on Mahuika and Cascade, built in-code profiling, and removed several performance bottlenecks. Sixteen pull requests have been merged upstream. The combined changes roughly halve the time to solution on QuakeCore's production rupture-source cases, and SW4 weak-scales with an efficiency of 1.10 from 1 to 8 Cascade nodes (1536 cores).

1 Scope and tasks

The consultancy was set up with the following tasks:

1. Confirm the observed performance of SW4 in single versus double precision and find the performance hotspots.
2. Review the single-precision port and validate that it is technically sound and suitable for production use.
3. Work with the project researchers to profile SW4, identify bottlenecks, and investigate why single-precision runs are slower than expected.
4. Agree on a performance improvement strategy, implement it, test the changes and report the new performance.
5. Evaluate code changes, build options and runtime configurations to identify the optimal settings for SW4 on HPC systems.

Work was coordinated between REANNZ and QuakeCore in weekly meetings. All code changes were submitted as pull requests against `ucgmsim/sw4 single_precision`, reviewed and checked by the project's CI (regression tests and `clang-tidy`).

2 Summary of accomplishments

- **Single-precision port validated.** The port now builds with Clang and GCC via a CMake switch (`-DUSE_DOUBLE=OFF`); test tolerances were adjusted so the test suite passes in single precision. Single precision is 1.3–1.9× faster than double, with errors that stay at the level of the discretisation error except near mesh-refinement interfaces (Section 3).
- **Best build configuration found.** On Mahuika Genoa nodes the classic Intel compiler with AVX-512 (`-march=icelake-server -qoverride-limits -qopt-zmm-usage=high`) is fastest, 27–29% ahead of GCC 15 and 2–5× ahead of the previous `gimkl`/AOCC builds (Section 4).

- **Built-in profiling.** SW4 now reports hierarchical region timers with MPI and OpenMP (OMPT) costs at the end of every run, with no need for external tools such as TAU (Section 5).
- **Kernel and algorithmic speedups.** A source time-function cache, constant-bound loops in the upper SBP closure, a reordered `Sarray::assign`, halo-only zeroing and j-tiling of the Cartesian RHS kernels. Individually these give between a few percent and 1.8×; on a 30-event Darfield production campaign the run time fell from about 3 h to 1.5 h (Section 6).
- **Bugs fixed.** Chunked float I/O in `Parallel_IO`, a missing reduction in the interface Jacobi solver, and several build portability problems (Section 7).
- **Runtime configuration and scaling on Cascade.** Pinning one MPI rank per 8-core CCD with 8 OpenMP threads is optimal; hyperthreading costs 31%. Weak-scaling efficiency is 1.10 at 8 nodes (Section 8).

3 Validation of the single-precision port

The `single_precision` branch did not compile with Clang 21 on macOS because of mixed float/-double arithmetic: calls such as `pow(x,2)` promote float arguments to double, creating ambiguous overloads and, more importantly, silent conversions in inner loops. These were fixed (PR #2) and a CMake single/double switch was added. The test tolerances were relaxed where the differences between single and double were consistent with round-off.

Twilight-zone (manufactured solution) and Lamb problem tests show no visible differences between single and double for tests without mesh refinement (`flat-twi-1`, `lamb-1`, `tw-att-1`). With mesh refinement (`refine-att-1`, `refine-el-1`) the single-precision error is noticeably larger (Fig. 1). The parts of the code that require double precision (the interface Jacobi iteration and mass-block inversion) were identified as candidates to remain in double.

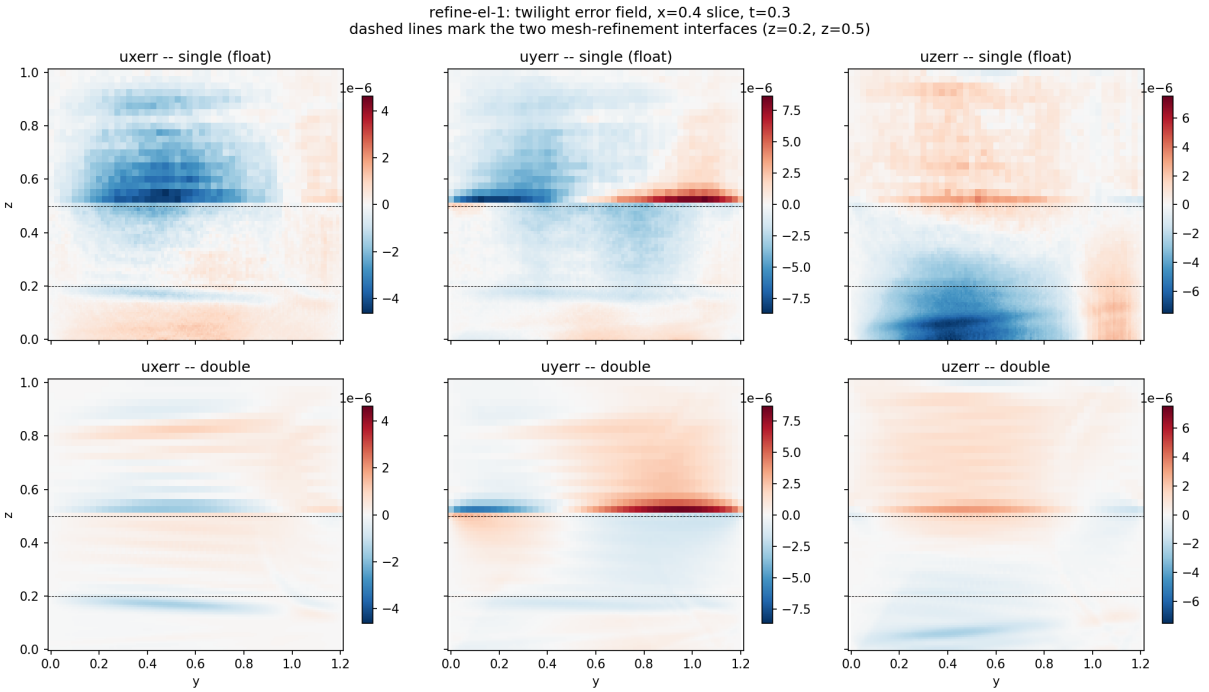


Figure 1: Twilight error field for `refine-el-1` on the slice $x = 0.4$ at $t = 0.3$, single (top) versus double (bottom) precision. The dashed lines mark the two mesh-refinement interfaces. The single-precision error has the same amplitude but carries visible round-off noise, concentrated around the refinement interfaces.

On the `gausshill-att-3` case (curvilinear grid, attenuation, mesh refinement) no difference between the single and double solutions is visible, and single precision is 1.6–1.9× faster.

Sensitivity of rupture sources. For QuakeCore's rupture-source cases, the forcing at a grid point is the sum of about 1200 point-source contributions that largely cancel. In float32 this makes the station output sensitive to round-off in the source time functions at the per-cent level. Comparing against a double-precision reference showed that reducing the magnitude of the time argument of the source splines decreased the single-precision error by a factor of ∼20 (Fig. 2). This change could not be merged because it conflicts with the prefilter pre-roll introduced upstream at the same time (PR #33), but it highlights where single precision is fragile.

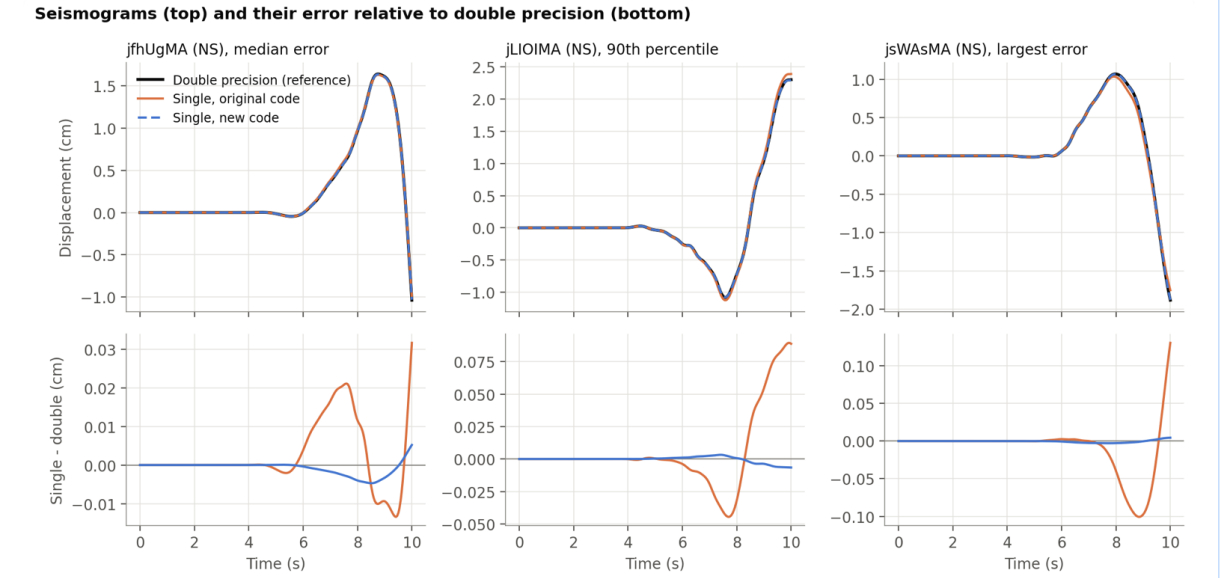


Figure 2: Station seismograms (top) and their error relative to double precision (bottom) for three stations of the `input_k4` rupture case: original single-precision code (orange) and with trimmed source splines (blue).

4 Build configuration

4.1 Portability

The CMake build was extended to work with the Mahuika toolchains (`gimkl`, `intel`, `foss`, AOCC with OpenMPI) (PR #3). This required restricting the MPI search to C/C++ (AOCC's `flang` cannot read `gfortran` module files), a configurable number of OpenMP threads per test to avoid oversubscription, and an `hpc3-genoa` target that falls back to `znver3`+AVX-512 when the compiler lacks `znver4`. Further changes switched the build to C++17 (PR #6), removed redundant `#pragma ivdep` next to `#pragma omp simd` that broke oneAPI builds (PR #17), and made CMake require a parallel HDF5 when `USE_HDF5=ON` (PR #31).

4.2 Compiler and target comparison

Fig. 3 compares toolchains on `gausshill-att-3`. Initially (generic targets) Intel was fastest on most tests, `gimkl` 1.0–1.45× slower and AOCC 1.4–2.1× slower, with the largest gaps on tests with attenuation. Targeting the Genoa architecture improved the Intel build by ∼1.2×, mostly in the curvilinear stencil (`curvilinear4sg`, 1.45× from SSE2 to AVX2).

The decisive step was enabling 512-bit vectors with the classic Intel compiler. Zen 4 implements a subset of AVX-512 which `-march=icelake-server` targets; `-qoverride-limits` is

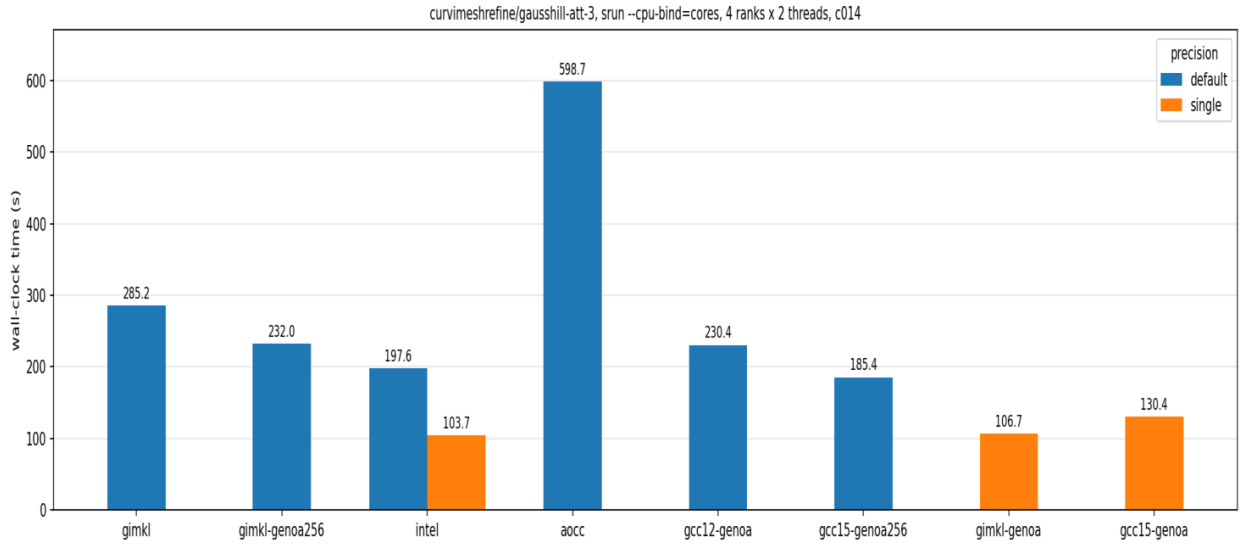


Figure 3: Wall-clock time of `gausshill-att-3` for different toolchains and targets (4 ranks × 2 threads, Genoa node).

needed to stop the compiler giving up on 4 of the 5 loops in `curvilinear4sg_ci_impl`, and `-qopt-zmm-usage=high` to use full-width registers. Table 1 shows the resulting single-precision timings.

Table 1: `gausshill-att-3`, single precision, 4 ranks × 2 threads, 4 runs per build on the same Genoa node (range in brackets).

build	time stepping (s)	wall (s)	wall vs Intel
Intel `icpc`, AVX-512	71.3 (69.2–73.2)	82.8 (79.1–85.1)	–
GCC 15.2, `-march=znver4`, 512-bit	101.3 (100.5–102.3)	104.9	+27%
GCC 15.2, `-march=znver4`, 256-bit	99.2 (97.8–101.4)	106.8	+29%

For comparison, the same case took over 200 s with `gimkl` and about 360 s with AOCC. The recommended build is

```
module purge; module load intel/2022a
cmake -DCMAKE_BUILD_TYPE=Release -DBUILD_SW4MOPT=OFF \
      -DCMAKE_C_COMPILER=mpiicc -DCMAKE_CXX_COMPILER=mpiicpc \
      -DCMAKE_Fortran_COMPILER=ifort \
      -DSW4_TARGET=hpc3-genoa -DUSE_DOUBLE=OFF ..
```

oneAPI container. SW4 was also built with Intel oneAPI (`icpx`) inside an Apptainer container. With controlled pinning, the container build runs at the same speed as the native `icpc` build (ratio 1.002 on `lamb-3`, 450.8 versus 449.7 s), with the same accuracy. Apparent gaps of 5–33% seen earlier were caused by rank placement, not by the compiler or the container.

5 Profiling infrastructure

Profiling with TAU on Mahuika was used in the first weeks. To make profiling routine for the researchers, hierarchical region timers were added directly to SW4 (PRs #36, #37):

- `SW4_TIME("name")` scoped timers for the main phases, the predictor/corrector sub-steps, output, `enforceIC` and the interface solve, and each attenuation mechanism in `evalRHS`;

- an end-of-run table of min/avg/max over ranks, optionally written as CSV (`SW4_TIMERS_CSV`);
- PMPI wrappers that charge MPI time, bytes and messages to the innermost region (CMake option `SW4_TIMERS_MPI`);
- an OMPT tool that measures per-thread busy time and therefore OpenMP load imbalance and fork/join overhead.

The timers revealed, for example, that `enforceIC` launched about 180 small parallel regions per time step with 77–86% thread idle time, and that the forcing dominated the rupture-source cases. They were used to produce the phase breakdowns in Section 8.

6 Performance optimisations

Table 2 lists the code optimisations. All are bit-identical to the code they replace unless stated otherwise.

Table 2: Code optimisations. Speedups are for the stated region or case.

PR	change	effect
#5	`Sarray::assign`: loop over (k, j) then i when reordering a Fortran-order array	part of `lamb-3`: 1026 → 431 s
#7	`rhs4th3fort(sgstr)_ci`: zero only the halo of `lu`, not the whole array	fewer memory sweeps
#14	Cartesian RHS kernels: tile the interior loop in j so the k-stencil stays in L2	Cartesian RHS
#13	Exact Lamb solution evaluated on the surface plane only	faster Lamb tests
#34	Pass grid point source vectors by `const` reference (were copied every call)	forcing
#41[†]	Evaluate each distinct source time function once per `Force`/`Force_tt` call	`input_k4`: 66 min → 40 min (1.65×), memory 19.4 → 16.4 GB
#38	Constant-bound loops in the upper SBP closure of the curvilinear and anisotropic kernels	curvilinear RHS 2.3×; `k4_c8` 1.8×

[†] Merged by J. Faulkner, combining the cache (commits `0140da08`, `8780d28e`) with tabulation of the prefiltered time functions.

Source time-function cache. QuakeCore's rupture models contain tens of thousands of sub-fault sources which are distributed to millions of grid-point sources, but only about one in a hundred has a distinct time function. Evaluating each distinct function once per call reduced the forcing on the `input_k4` case from 318 s to 2.5 s. In production, the QuakeCore team reported that this optimisation reduced the run time of the Darfield campaign (162 runs, some on curvilinear grids) from about 3 h to 1.5 h per run, as the forcing on the ranks overlapping the fault was holding up all other ranks.

Upper SBP closure. The curvilinear stencil's upper boundary closure ($k = n_k - 5, \ldots, n_k$) ran 4.5× slower per point than the lower closure because its inner loops had run-time bounds (`q = nk-7 .. nk`), which prevented the compiler from unrolling and vectorising them efficiently. Rewriting them with constant trip counts made both closures equally fast. Because QuakeCore's curvilinear grids are thin (13–14 planes), the closure covers ∼45% of the curvilinear points, so the curvilinear RHS dropped from 38.1 to 16.8 s and the `k4_c8` test case from 113.9 to 63.1 s.

Work not (yet) merged.

- *Fused attenuation mechanisms* (branch `fuse-attenuation-mechanisms`): one sweep for all attenuation mechanisms in the curvilinear RHS. Bit-identical, but only 4.6% faster, since that kernel is compute bound.
- *Merged interface Jacobi kernels* (branch `enforceic-jacobi-merge`): the four Jacobi kernels in `consintp` run in a single parallel region. Jacobi -45%, `enforceIC` -23%, time step $\sim -3\%$. Also fixes the residual bug below.
- *Shorter prefilter precursor* (`precursor_efolds` option): 8 instead of 12 e-folds saves 21% of time steps, but changes weak traces by up to 40%. The plan, currently, is to keep the precursor but at the same time investigate filtering as a post-processing step.

7 Bug fixes

- **Parallel I/O** (PR #35): the chunked float read and write paths in `Parallel_IO` were broken for large single-precision transfers.
- **Interface Jacobi residual**: the `omp simd` loops updated the max-norm residual without a reduction clause (undefined behaviour), giving a wrong residual on multiple ranks. Fixed on `enforceic-jacobi-merge`.
- **Tests**: the old unit tests were reinstated (`ctest -L unit`) and the `lamb-1` and attenuation reference files regenerated (PRs #12, #32).

8 Runtime configuration and scaling

8.1 Process binding

Many apparent performance differences early in the project were caused by rank placement rather than by the code. The recommended settings with Slurm are

```
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PROC_BIND=close
export OMP_PLACES=cores
srun --mpi=pmix --cpu-bind=cores sw4 input.in
```

(`--mpi=pmi2` for Intel MPI and OpenMPI 4). With OpenMPI and `mpiexec`, `--map-by slot:PE=N --bind-to core` is required, otherwise ranks are left unbound and threads pile up on the same cores.

8.2 MPI/OpenMP balance

On a 64-core Cascade allocation (`small` case, 242 steps), the time stepping is 23–30% slower with pure MPI (64×1) than with 2–8 threads per rank; 16×4 is fastest, within 5% of 32×2 and 8×8. For short runs, however, a phase outside time stepping grows as the number of ranks decreases, so total wall time favours fewer threads.

On full Cascade nodes (2 sockets, 24 CCDs of 8 cores sharing an L3 cache), the best layout is one rank per CCD, i.e. 24 ranks × 8 threads per node, with every rank's threads confined to its CCD (Table 3). SW4 is memory bound, and hyperthreading makes it 31% slower.

8.3 Strong and weak scaling

Strong scaling on one Cascade node (`input_k4_c4`, 2 threads per rank) gives a speedup of 2.96 going from 6 to 24 ranks (74% efficiency).

For weak scaling the grid spacing was refined from 800 m to 400 m so that each node holds about 44 M grid points, from 1 to 8 nodes (192 ranks, 1536 cores). The time per grid point per step per node falls from 11.33 to 10.28 ns, an efficiency of 1.10 (Fig. 4). The solver alone has an

Table 3: Rank/thread layouts on 8 Cascade nodes (351 M grid points, 4166 steps).

layout per node	ranks	time stepping (s)	ns/point/step/node	vs 24×8
24 × 8	192	1879.8	10.28	–
24 × 16, hyperthreads	192	2458.0	13.44	+31%
48 × 4	384	1922.6	10.51	+2.3%

efficiency of 1.05. Efficiency exceeds 1 because some work does not grow with the node count: the rupture source has a fixed number of subfaults and the surface images grow with area rather than volume. Communication stays at about 17% of the step time at all node counts, half of which is waiting for slower ranks. The largest remaining imbalance is in the forcing: at 8 nodes the busiest rank spends about 43× the average time on it, since only ranks overlapping the fault do source work.

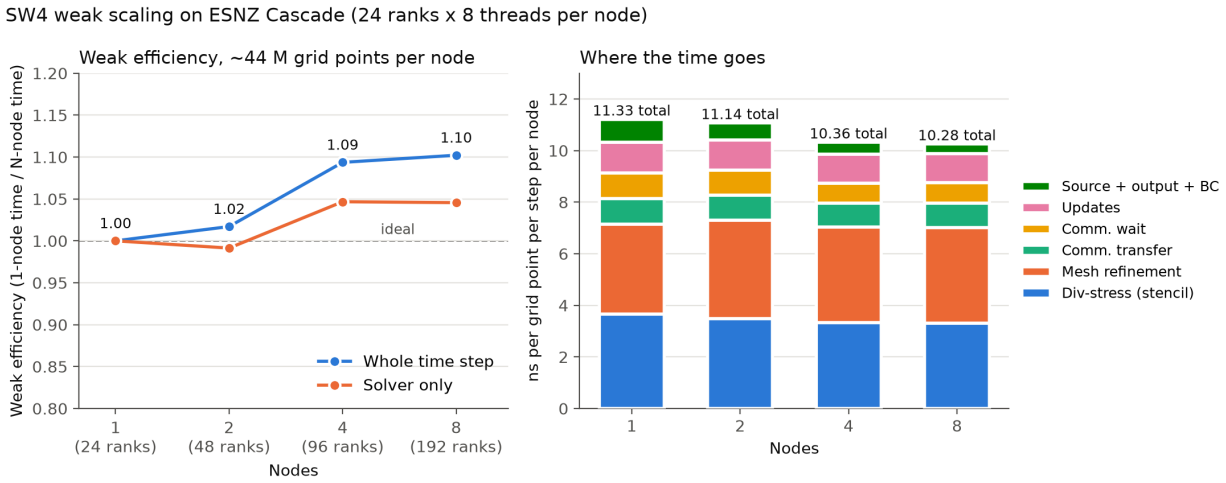


Figure 4: Weak scaling of SW4 on Cascade, 24 ranks × 8 threads per node. Left: efficiency of the whole time step and of the solver alone. Right: time per grid point per step per node by phase.

9 Pull requests merged upstream

In addition, the source time-function cache was merged as part of PR #41 (16 Oct).

10 Next steps

- Contribute some or all of the enhancements back to the original SW4 code (`geodynamics/sw4`), so that the wider SW4 community benefits and QuakeCore's fork does not drift further from upstream. Candidates include the build portability fixes, the region timers, the bug fixes, and the precision-independent kernel optimisations (SBP closure, source time-function cache, `Sarray::assign`, RHS tiling).
- Open pull requests for the merged interface Jacobi kernels (with the residual fix) and decide on the fused attenuation kernel.
- Investigate the decomposition of irregular domains such as the Darfield model, and map the domain decomposition onto the CCD layout.
- Reduce the forcing imbalance on ranks overlapping the fault.
- Vectorise the anisotropic curvilinear kernels, which currently do not vectorise at all.

PR	merged	description
#2	30 Sep	Single-precision build fixes (Clang 21, `pow` promotion), test tolerances
#3	30 Sep	Multi-toolchain CMake build on Mahuika, AVX-512 flags for `icpc` on Genoa
#5	1 Oct	`Sarray::assign` loop reordering
#6	5 Oct	Build with C++17
#7	8 Oct	Zero only the halo in `rhs4th3fort(sgstr)_ci`
#12	8 Oct	Regenerate attenuation reference files
#13	8 Oct	Exact Lamb solution on the surface plane only
#14	8 Oct	j-tiling of the Cartesian RHS kernels
#17	9 Oct	Remove `#pragma ivdep` next to `#pragma omp simd`
#31	9 Oct	CMake: require a parallel HDF5
#32	12 Oct	Reinstate unit tests (`ctest -L unit`)
#34	13 Oct	Pass point-source vectors by `const` reference
#36	14 Oct	Hierarchical region timers, MPI and OMPT profiling
#35	16 Oct	`Parallel_IO` chunked float read/write fix
#37	16 Oct	Pre-roll progress; per-mechanism attenuation timers
#38	16 Oct	Constant-bound loops in the upper SBP closure

- Reduce start-up and output cost (HDF5 paging already reduced start-up from 30 to 1 min on one case).
- Repeat the scaling study on the Darfield input and on CyberShake-size sources.